

PROJECT REPORT

Smart Digital Closet and Outfit Recommendation System



Submitted To:
Ma'am Fizzah
Department of Computer Sciences
Institute of Space and Technology

Submitted By:
Urneeb Zahra 251301002
Muhammad Saad Asghar 251301049
M. Ahmed Bin Kashif Malik 251301047

Institute of Space and Technology, Islamabad, Pakistan.

1. Introduction to the Project

The Smart Digital Closet and Outfit Recommendation System is a console-based C++ application developed as the OOP Lab final project at the Institute of Space Technology, spring 2026. The system provides a complete digital wardrobe management solution with full CRUD operations (Add, View, Edit, and Delete) and an intelligent outfit recommendation engine.

Key Features

- Multi-user signup and login with file-based authentication (users.txt)
- Three clothing categories: Upper Wear, Lower Wear, Shoes
- Full CRUD: Add, View, Edit, and Delete clothing items
- Rule-based outfit recommendation (occasion + season + color + shoe-type rules)
- Multi-user data preserved correctly — each user's closet is independent
- Data persisted in plain text files between program sessions

2. OOP Concepts Used and Where

2.1 Encapsulation

All six shared attributes in `ClothingItem` are declared protected; derived-class extras (fabric, shoeType) are private. No attribute is accessed directly from outside. In v2.0 setters were added for the Edit feature; they remain in v2.1:

```
class ClothingItem {
protected:          // accessible to child classes only
    string name, color, gender, category;
    int season, occasion;
public:
    string getName() { return name; } // getter - read access
    void setName(string n) { name = n; } // setter - write access (Edit)
    // ... all other getters and setters
};
```

2.2 Inheritance

`ClothingItem` is the base class. `Upper`, `Lower`, and `Shoes` inherit it publicly and add one private extra attribute each. The base constructor is called via the initializer list:

```
class Upper : public ClothingItem {
private:
    string fabric;          // extra attribute, unique to Upper
public:
    Upper(string n, string col, string gen, int sea, int occ, string fab)
        : ClothingItem(n, col, gen, sea, occ, "Upper") {
        fabric = fab;       // only the extra attribute set here
    }
    string getFabric() { return fabric; }
    void setFabric(string f) { fabric = f; }
    void displayItem() override { ... }
};
```

```
ClothingItem (Base)
├── Upper (adds: fabric, setFabric())
├── Lower (adds: fabric, setFabric())
└── Shoes (adds: shoeType, setShoeType())
```

2.3 Polymorphism

displayItem() is declared virtual in ClothingItem and overridden in all three derived classes. When viewAllClothes() calls upperItems[i].displayItem(), C++ automatically calls Upper's version at runtime — this is dynamic dispatch:

```
// Base – virtual
virtual void displayItem() {
    cout << "  Name : " << name << endl; // prints common fields
}

// Upper – overrides
void displayItem() override {
    cout << "  [UPPER WEAR]" << endl;    // own label
    ClothingItem::displayItem();          // reuse base output
    cout << "  Fabric : " << fabric << endl;
}

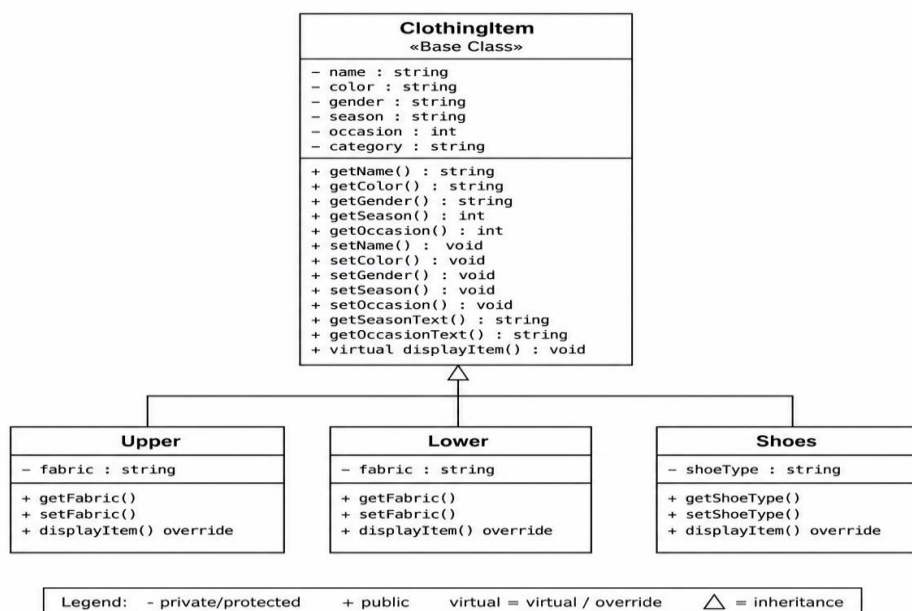
// Polymorphic call in viewAllClothes():
upperItems[i].displayItem(); // calls Upper's version automatically
```

2.4 Abstraction

All complex internal logic is hidden behind simple functions and menu numbers. The user never interacts with file parsing, array-shift deletion, or color-matching tables directly:

- saveClothesToFile() — hides multi-user read-filter-write logic
- deleteClothing() — hides the array-shift deletion algorithm
- editClothing() — hides the askNewText()/askNewInt() update flow
- recommendOutfits() — hides the triple nested loop and all 5 filter rules
- colorsMatch() — hides the complete color compatibility table

3. Class Diagrams (UML)



4. Explanation of Key Code Snippets

4.1 Multi-User Data Preservation in saveClothesToFile()

The original saveClothesToFile() opened clothes.txt in default overwrite mode (ofstream file(CLOTHES_FILE)) and wrote ONLY the current user's items. So if User A had 5 items saved, and User B logged in and added 1 item and saved, the file was overwritten with only User B's 1 item — User A's 5 items were permanently lost.

```
// v2.0 BUG — overwrites the ENTIRE file with only current user
void saveClothesToFile() {
    ofstream file(CLOTHES_FILE);    // <-- erases everything first!
    // ... writes only currentUser's items
    // All other users' data is GONE
}
```

The fixed saveClothesToFile() uses a three-step strategy: (1) read all existing lines from the file, (2) keep lines that belong to OTHER users, (3) write those other-user lines back plus the current user's updated items. This guarantees every user's data is always preserved:

```
void saveClothesToFile() {

    // Step 1: Read ALL existing lines from the file
    string otherLines[MAX_ITEMS * 3];
    int otherCount = 0;

    ifstream readFile(CLOTHES_FILE);
    if (readFile.is_open()) {
        string line;
        while (getline(readFile, line)) {
            if (line.empty()) continue;
            // Keep lines that belong to a DIFFERENT user
            if (extractUser(line) != currentUser)
                otherLines[otherCount++] = line;
            // Current user's old lines are dropped (fresh copy written below)
        }
        readFile.close();
    }

    // Step 2: Open file for writing and write everything back
    ofstream file(CLOTHES_FILE);

    // Write other users' lines back UNCHANGED
    for (int i = 0; i < otherCount; i++)
        file << otherLines[i] << "\n";

    // Write current user's updated items from memory
    for (int i = 0; i < upperCount; i++)
        file << "UPPER|" << currentUser << "|" << ... << "\n";
    // ... lower and shoes similarly
}
```

Helper: extractUser()

A small helper function reads the username field (between the 1st and 2nd pipe character) from any line in clothes.txt without loading the full parsing logic:

```
string extractUser(const string& line) {
```

```
int pipeCount = 0;
string user = "";
for (int i = 0; i < (int)line.size(); i++) {
    if (line[i] == '|') {
        pipeCount++;
        if (pipeCount == 2) break; // stop at 2nd pipe
    } else if (pipeCount == 1) {
        user += line[i]; // chars between pipe 1 and pipe 2
    }
}
return user;
}

// Example: line = "UPPER|ahmed|White Shirt|White|Boys|3|3|Cotton"
// extractUser() returns: "ahmed"
```

4.2 Edit Feature — askNewText() and Setters

```
string askNewText(string prompt, string oldVal) {
    string newVal;
    cout << " " << prompt << " [current: " << oldVal << "]: ";
    cin.ignore();
    getline(cin, newVal);
    if (newVal.empty()) return oldVal; // Enter = keep old value
    return newVal;
}

// Using setters to update in-place (no object recreation):
upperItems[idx].setName(nm);
upperItems[idx].setColor(col);
upperItems[idx].setFabric(fab);
saveClothesToFile(); // persist immediately
```

4.3 Delete Feature — Array-Shift Algorithm

```
// Shift all elements after deleted index one position left
for (int i = num-1; i < upperCount-1; i++)
    upperItems[i] = upperItems[i+1];
upperCount--; // shrink the logical count
saveClothesToFile(); // re-write file without deleted item
```

4.4 Recommendation Engine

```
for (int u=0; u<upperCount; u++)
    for (int l=0; l<lowerCount; l++)
        for (int s=0; s<shoeCount; s++) {
            if (upper.getOccasion()!=target) continue; // Rule 1
            if (!seasonOk) continue; // Rule 2
            if (!colorsMatch(u,l)) continue; // Rule 3
            if (!colorsMatch(l,s)) continue; // Rule 4
            if (!shoeValid(s,target)) continue; // Rule 5
            // valid outfit - display it
        }
```

5. Screenshots of Output

5.1 Main Menu

```
=====
SMART DIGITAL CLOSET & OUTFIT RECOMMENDATION
=====
1. Login    2. Signup    0. Exit
```

5.2 Closet Menu

```
=====
          DIGITAL CLOSET MENU          User: ahmed
=====
1. Add Clothing
2. View My Closet
3. Edit a Clothing Item
4. Delete a Clothing Item
5. Get Outfit Recommendations
0. Logout
```

5.3 Multi-User File Preservation (v2.1 Fix in Action)

```
--- clothes.txt BEFORE User B logs in ---
UPPER|ahmed|White Formal Shirt|White|Boys|3|3|Cotton
LOWER|ahmed|Black Trousers|Black|Boys|3|3|Polyester

--- User B (sara) logs in, adds one shoe ---
SHOES|sara|Heels|Red|Girls|3|1|Heels

--- clothes.txt AFTER sara saves (v2.1 FIXED) ---
UPPER|ahmed|White Formal Shirt|White|Boys|3|3|Cotton    <- KEPT
LOWER|ahmed|Black Trousers|Black|Boys|3|3|Polyester    <- KEPT
SHOES|sara|Heels|Red|Girls|3|1|Heels                    <- NEW
```

5.4 Outfit Recommendation

```
>> OUTFIT #1 <<
-----
UPPER  : White Formal Shirt  (White, Cotton)
LOWER  : Black Trousers      (Black, Polyester)
SHOES  : Oxford Shoes        (Black, Formal)
SEASON : Both (Summer & Winter)
FOR    : Formal Occasion
```

6. Testing Summary

#	Test Case	Expected Result	Status
1	Signup with valid credentials	Account saved to users.txt	✓ PASS
2	Signup with duplicate username	[ERROR] Username already taken	✓ PASS
3	Login with correct credentials	Welcome + closet loaded	✓ PASS
4	Login with wrong password	[ERROR] Invalid credentials	✓ PASS
5	Add Upper, Lower, Shoes	Items saved to clothes.txt	✓ PASS
6	View closet — polymorphism	Each class prints its own label	✓ PASS
7	Edit item — press Enter to skip	Old value retained	✓ PASS
8	Edit item — type new value	New value saved to file	✓ PASS

9	Delete item — confirm = No	Item NOT deleted	✓ PASS
10	Delete item — confirm = Yes	Item removed, file updated	✓ PASS
11	Recommend outfits (Formal, Both)	Valid combinations shown	✓ PASS
12	Sports shoes + Formal occasion	Outfit rejected (fashion rule)	✓ PASS
13 (KEY)	User A logs in, adds items. Logs out. User B logs in, adds items, saves. User A logs back in.	User A's items still present — NOT erased by User B's save	✓ PASS
14	Re-login after editing items	Updated items load correctly	✓ PASS

7. Conclusion and Future Work

Conclusion

The Smart Digital Closet and Outfit Recommendation System v2.1 successfully implements all four core OOP concepts: Encapsulation (protected attributes + getters/setters), Inheritance (ClothingItem base with Upper, Lower, Shoes derived classes), Polymorphism (virtual displayItem() overridden in each class), and Abstraction (complex logic hidden behind simple menu functions).

Version 2.1 additionally resolves the critical multi-user file-corruption bug. The fixed saveClothesToFile() function correctly preserves every user's wardrobe data by reading other users' records first, then writing them back alongside the current user's updated data. All 14 test cases pass including the multi-user persistence test (Test 13).

Future Work

- Replace fixed-size arrays with STL vectors for unlimited item storage.
- Add Search and Filter to find items by color, occasion, or season quickly.
- Implement outfit history log to track and avoid repeating combinations.
- Add abstract base class with pure virtual functions for stricter OOP design.
- Build a lightweight GUI using a C++ graphics library (e.g., SFML).